

```
CREATE TABLE Locations (  
    location_id INT AUTO_INCREMENT PRIMARY KEY, -- Primary Key with auto-increment  
    name VARCHAR(100) NOT NULL,  
    address VARCHAR(255),  
    region VARCHAR(50)  
);
```

```
CREATE TABLE Supplies (  
    supply_id INT AUTO_INCREMENT PRIMARY KEY, -- Primary Key with auto-increment  
    name VARCHAR(100) NOT NULL,  
    category VARCHAR(50),  
    unit VARCHAR(20),  
    reorder_level INT NOT NULL  
);
```

```
CREATE TABLE Stock (  
    supply_id INT NOT NULL,  
    location_id INT NOT NULL,  
    quantity INT NOT NULL,  
    expiration_date DATE,  
    PRIMARY KEY (supply_id, location_id), -- Composite primary key  
    FOREIGN KEY (supply_id) REFERENCES Supplies(supply_id) ON DELETE CASCADE,  
    FOREIGN KEY (location_id) REFERENCES Locations(location_id) ON DELETE CASCADE  
);
```

```
CREATE TABLE Suppliers (  
    supplier_id INT AUTO_INCREMENT PRIMARY KEY, -- Primary Key with auto-increment
```

```
name VARCHAR(100) NOT NULL,  
contact_info VARCHAR(100),  
lead_time INT  
);
```

```
CREATE TABLE Supply_Orders (  
    order_id INT AUTO_INCREMENT PRIMARY KEY, -- Primary Key with auto-increment  
    supplier_id INT NOT NULL, -- Foreign Key referencing Suppliers  
    supply_id INT NOT NULL, -- Foreign Key referencing Supplies  
    order_date DATE NOT NULL,  
    quantity_ordered INT NOT NULL,  
    status VARCHAR(50),  
    FOREIGN KEY (supplier_id) REFERENCES Suppliers(supplier_id),  
    FOREIGN KEY (supply_id) REFERENCES Supplies(supply_id)  
);
```

```
CREATE TABLE Staff (  
    staff_id INT AUTO_INCREMENT PRIMARY KEY, -- Primary Key with auto-increment  
    name VARCHAR(100) NOT NULL,  
    role VARCHAR(50),  
    contact_info VARCHAR(100),  
    assigned_location_id INT,  
    FOREIGN KEY (assigned_location_id) REFERENCES Locations(location_id)  
);
```

```
CREATE TABLE Emergency_Events (  
    event_id INT AUTO_INCREMENT PRIMARY KEY, -- Primary Key with auto-increment  
    event_name VARCHAR(100) NOT NULL,  
    start_date DATE NOT NULL,  
    end_date DATE,
```

```
    event_type VARCHAR(50),  
    description TEXT  
);
```

```
CREATE TABLE Emergency_Distributions (  
    distribution_id INT AUTO_INCREMENT PRIMARY KEY, -- Primary Key with auto-increment  
    supply_id INT NOT NULL, -- Foreign Key referencing Supplies  
    location_id INT NOT NULL, -- Foreign Key referencing Locations  
    event_id INT NOT NULL, -- Foreign Key referencing Emergency_Events  
    quantity_dispatched INT NOT NULL,  
    date_dispatched DATE NOT NULL,  
    emergency_type VARCHAR(50),  
    FOREIGN KEY (supply_id) REFERENCES Supplies(supply_id),  
    FOREIGN KEY (location_id) REFERENCES Locations(location_id),  
    FOREIGN KEY (event_id) REFERENCES Emergency_Events(event_id)  
);
```

```
CREATE TABLE Event_Supplies (  
    event_id INT NOT NULL,  
    supply_id INT NOT NULL,  
    quantity_needed INT NOT NULL,  
    PRIMARY KEY (event_id, supply_id),  
    FOREIGN KEY (event_id) REFERENCES Emergency_Events(event_id) ON DELETE CASCADE,  
    FOREIGN KEY (supply_id) REFERENCES Supplies(supply_id) ON DELETE CASCADE  
);
```

```
CREATE TABLE Staff_Events (  
    staff_id INT NOT NULL,  
    event_id INT NOT NULL,
```

```

    role VARCHAR(50),

    PRIMARY KEY (staff_id, event_id),

    FOREIGN KEY (staff_id) REFERENCES Staff(staff_id) ON DELETE CASCADE,

    FOREIGN KEY (event_id) REFERENCES Emergency_Events(event_id) ON DELETE CASCADE

);

CREATE TABLE Supplier_Locations (

    supplier_id INT NOT NULL,

    location_id INT NOT NULL,

    delivery_frequency INT, -- Optional: e.g., deliveries per month

    PRIMARY KEY (supplier_id, location_id),

    FOREIGN KEY (supplier_id) REFERENCES Suppliers(supplier_id) ON DELETE CASCADE,

    FOREIGN KEY (location_id) REFERENCES Locations(location_id) ON DELETE CASCADE

);

-- Insert data into Locations table

INSERT INTO Locations (name, address, region)

VALUES

('Central Warehouse', '123 Main St, City A', 'North'),

('East Storage', '456 Elm St, City B', 'East'),

('West Depot', '789 Oak St, City C', 'West'),

('South Hub', '321 Pine St, City D', 'South'),

('North Facility', '654 Maple St, City E', 'North'),

('Midtown Storage', '987 Cedar St, City F', 'Central'),

('City Center Warehouse', '456 Ash St, City H', 'Central'),

('Emergency Depot', '789 Birch St, City I', 'South'),

('Regional HQ', '123 Willow St, City G', 'East'),

('Coastal Facility', '321 Palm St, City J', 'Coastal');

-- Insert data into Supplies table

INSERT INTO Supplies (name, category, unit, reorder_level)

```

VALUES

('First Aid Kit', 'Medical', 'Box', 50),
('Water Bottles', 'Beverage', 'Liters', 200),
('Blankets', 'Shelter', 'Piece', 100),
('Canned Food', 'Food', 'Cans', 150),
('Flashlights', 'Lighting', 'Piece', 30),
('Batteries', 'Power', 'Pack', 50),
('Tents', 'Shelter', 'Piece', 10),
('Fire Extinguishers', 'Safety', 'Piece', 20),
('Medicines', 'Medical', 'Box', 40),
('Emergency Radios', 'Communication', 'Piece', 15);

-- Insert data into Stock table

INSERT INTO Stock (supply_id, location_id, quantity, expiration_date)

VALUES

(1, 1, 100, '2025-01-01'),
(2, 2, 200, '2024-12-01'),
(3, 3, 50, NULL),
(4, 4, 120, '2025-03-01'),
(5, 5, 70, '2024-11-01'),
(6, 6, 80, '2025-06-01'),
(7, 7, 30, NULL),
(8, 8, 40, '2026-01-01'),
(9, 9, 60, '2024-10-01'),
(10, 10, 20, '2025-08-01');

-- Insert data into Suppliers table

INSERT INTO Suppliers (name, contact_info, lead_time)

VALUES

```
('Global Health Supplies', 'health@supplies.com', 7),  
( 'WaterWorks Co.', 'contact@waterworks.com', 10),  
( 'Food Essentials', 'order@foodessentials.com', 5),  
( 'Light and Power', 'support@lnp.com', 8),  
( 'TentPro', 'sales@tentpro.com', 12),  
( 'Safety First Inc.', 'info@safetyfirst.com', 6),  
( 'MediSupply', 'admin@medisupply.com', 7),  
( 'RadioComm Ltd.', 'sales@radiocomm.com', 15),  
( 'Global Distributors', 'support@globaldist.com', 9),  
( 'North Supply Chain', 'info@northchain.com', 4);
```

```
-- Insert data into Supply_Orders table
```

```
INSERT INTO Supply_Orders (supplier_id, supply_id, order_date, quantity_ordered, status)  
VALUES  
(1, 1, '2024-11-01', 50, 'Pending'),  
(2, 2, '2024-10-15', 300, 'Completed'),  
(3, 3, '2024-09-20', 150, 'Completed'),  
(4, 4, '2024-11-05', 100, 'Pending'),  
(5, 7, '2024-10-10', 10, 'Completed'),  
(6, 8, '2024-09-25', 20, 'Completed'),  
(7, 9, '2024-10-30', 30, 'Pending'),  
(8, 10, '2024-11-10', 5, 'Pending'),  
(9, 5, '2024-10-15', 70, 'Completed'),  
(10, 6, '2024-11-20', 80, 'Pending');
```

```
-- Insert data into Staff table
```

```
INSERT INTO Staff (name, role, contact_info, assigned_location_id)  
VALUES  
( 'John Smith', 'Manager', 'john.smith@example.com', 1),
```

```
('Alice Brown', 'Logistics Coordinator', 'alice.brown@example.com', 2),  
( 'Bob Johnson', 'Technician', 'bob.johnson@example.com', 3),  
( 'Emily Davis', 'Inventory Specialist', 'emily.davis@example.com', 4),  
( 'Michael Wilson', 'Supervisor', 'michael.wilson@example.com', 5),  
( 'Jessica Taylor', 'Driver', 'jessica.taylor@example.com', 6),  
( 'Chris White', 'Warehouse Worker', 'chris.white@example.com', 7),  
( 'Sarah Hall', 'Emergency Response', 'sarah.hall@example.com', 8),  
( 'David Clark', 'Procurement Officer', 'david.clark@example.com', 9),  
( 'Anna Moore', 'Administrator', 'anna.moore@example.com', 10);
```

```
-- Insert data into Emergency_Events table
```

```
INSERT INTO Emergency_Events (event_name, start_date, end_date, event_type, description)
```

```
VALUES
```

```
('Hurricane Relief', '2024-11-01', '2024-11-10', 'Natural Disaster', 'Relief efforts for Hurricane Omega.'),  
( 'Earthquake Assistance', '2024-10-20', '2024-10-30', 'Natural Disaster', 'Aid provided after Earthquake Delta.'),  
( 'Flood Recovery', '2024-09-15', '2024-09-25', 'Natural Disaster', 'Supporting regions affected by heavy floods.'),  
( 'Wildfire Response', '2024-08-01', '2024-08-15', 'Natural Disaster', 'Distributing supplies for wildfire victims.'),  
( 'Pandemic Support', '2024-07-01', '2024-07-31', 'Health Crisis', 'Medical aid and resources during outbreak.'),  
( 'Drought Assistance', '2024-06-10', '2024-06-20', 'Natural Disaster', 'Water and food aid during drought.'),  
( 'Refugee Support', '2024-05-01', '2024-05-15', 'Humanitarian', 'Help for displaced individuals.'),  
( 'Storm Relief', '2024-04-01', '2024-04-10', 'Natural Disaster', 'Emergency response to storms.'),  
( 'Tsunami Relief', '2024-03-01', '2024-03-10', 'Natural Disaster', 'Efforts to aid tsunami victims.'),  
( 'Fire Evacuation Support', '2024-02-01', '2024-02-15', 'Safety', 'Evacuation and shelter during fire outbreak.');
```

```
-- Insert data into Emergency_Distributions table
```

```
INSERT INTO Emergency_Distributions (supply_id, location_id, event_id, quantity_dispatched,  
date_dispatched, emergency_type)
```

```
VALUES
```

```
(1, 1, 1, 20, '2024-11-02', 'Natural Disaster'),  
(2, 2, 2, 100, '2024-10-21', 'Natural Disaster'),  
(3, 3, 3, 50, '2024-09-16', 'Natural Disaster'),  
(4, 4, 4, 80, '2024-08-02', 'Natural Disaster'),  
(5, 5, 5, 30, '2024-07-02', 'Health Crisis'),  
(6, 6, 6, 40, '2024-06-11', 'Natural Disaster'),  
(7, 7, 7, 10, '2024-05-02', 'Humanitarian'),  
(8, 8, 8, 25, '2024-04-02', 'Natural Disaster'),  
(9, 9, 9, 60, '2024-03-02', 'Natural Disaster'),  
(10, 10, 10, 15, '2024-02-02', 'Safety');
```

```
-- Insert data into Event_Supplies table
```

```
INSERT INTO Event_Supplies (event_id, supply_id, quantity_needed)
```

```
VALUES
```

```
(1, 1, 25),  
(1, 2, 100),  
(2, 3, 50),  
(2, 4, 80),  
(3, 5, 60),  
(3, 6, 30),  
(4, 7, 10),  
(4, 8, 20),  
(5, 9, 40),  
(5, 10, 15);
```

```
-- Insert data into Staff_Events table
```

```
INSERT INTO Staff_Events (staff_id, event_id, role)
```

VALUES

(1, 1, 'Coordinator'),
(2, 2, 'Supervisor'),
(3, 3, 'Technician'),
(4, 4, 'Inventory Manager'),
(5, 5, 'Team Lead'),
(6, 6, 'Driver'),
(7, 7, 'Warehouse Worker'),
(8, 8, 'Emergency Responder'),
(9, 9, 'Procurement Specialist'),
(10, 10, 'Administrator');

-- Insert data into Supplier_Locations table

INSERT INTO Supplier_Locations (supplier_id, location_id, delivery_frequency)

VALUES

(1, 1, 4),
(2, 2, 2),
(3, 3, 1),
(4, 4, 3),
(5, 5, 5),
(6, 6, 2),
(7, 7, 4),
(8, 8, 3),
(9, 9, 1),
(10, 10, 6);

select * from Supplies;

select * from Suppliers ;

select * from Stock ;

select * from Locations ;

```
select * from Emergency_Distributions ;
```

```
select * from Emergency_Events ;
```

```
select * from Staff;
```

```
select * from Supply_Orders;
```

```
select * from Staff_Events ;
```

```
select * from Supplier_Locations ;
```

```
select * from Event_Supplies;
```

```
/* Query to list supplies with a reorder level less than 50:*/
```

```
SELECT name, reorder_level
```

```
FROM Supplies
```

```
WHERE reorder_level < 50;
```

```
/*Retrieve supplies that start with the letter 'F':*/
```

```
SELECT name FROM Supplies WHERE name LIKE 'F%';
```

```
/* Query to find stock entries with quantities between 20 and 200:*/
```

```
SELECT supply_id, location_id, quantity
```

```
FROM Stock
```

```
WHERE quantity BETWEEN 20 AND 200;
```

```
/* Find emergency events that happened after 2020:*/
```

```
SELECT event_name, start_date
```

```
FROM Emergency_Events
```

```
WHERE start_date > '2020-01-01';
```

```
/*Query to fetch emergency events of specific types:*/
```

```
SELECT event_id, event_name, event_type
```

```
FROM Emergency_Events
```

```
WHERE event_type IN ('Natural Disaster', 'Health Emergency');
```

```
/* List categories with an average reorder level greater than 50 */
```

```
SELECT category, AVG(reorder_level) AS "Average Reorder Level"  
FROM Supplies  
GROUP BY category  
HAVING AVG(reorder_level) > 50;
```

```
/* Count the total number of supplies*/
```

```
SELECT COUNT(*) AS "Total Supplies" FROM Supplies;
```

```
/* Calculate the average reorder level by category*/
```

```
SELECT category, AVG(reorder_level) AS "Average Reorder Level"  
FROM Supplies GROUP BY category;
```

```
/* Find Supplies That Are Below the Average Reorder Level*/
```

```
SELECT name, reorder_level FROM Supplies  
WHERE  
reorder_level < (SELECT AVG(reorder_level) FROM Supplies);
```

```
/* Total dispatched supplies for an event:*/
```

```
SELECT event_id, SUM(quantity_dispatched) AS "Total Dispatched"  
FROM Emergency_Distributions  
GROUP BY event_id;
```

```
/* Query to count the number of supplies in each category:*/
```

```
SELECT category, COUNT(*) as supply_count  
FROM Supplies  
GROUP BY category
```

```
ORDER BY supply_count DESC;
```

```
/* Sort supplies by name in ascending order:*/
```

```
SELECT * FROM Supplies ORDER BY name ASC;
```

```
/* Retrieve the name of the supply with the highest price*/
```

```
SELECT name, unit AS "Price"
```

```
FROM Supplies
```

```
WHERE unit = ( SELECT MAX(unit)
```

```
FROM Supplies);
```

```
/* find the most recently added supply*/
```

```
SELECT supply_id, name
```

```
FROM Supplies
```

```
WHERE supply_id = ( SELECT MAX(supply_id)
```

```
FROM Supplies);
```

```
/*To Get the earliest and latest stock expired dates:*/
```

```
SELECT MIN(expiration_date) AS earliest_expiry_date,
```

```
MAX(expiration_date) AS latest_expiry_date
```

```
FROM Stock;
```

```
/* List all supplies distributed during emergency events*/
```

```
SELECT
```

```
ee.event_name,
```

```
s.name AS "Supply Name",
```

```
ed.quantity_dispatched
```

```
FROM
```

```
Emergency_Distributions ed
```

JOIN

Emergency_Events ee ON ed.event_id = ee.event_id

JOIN

Supplies s ON ed.supply_id = s.supply_id;

/*Retrieve supplies and their current stock level for each location:*/

SELECT l.name AS "Location", s.name AS "Supply", st.quantity

FROM Stock st

JOIN Supplies s ON st.supply_id = s.supply_id

JOIN Locations l ON st.location_id = l.location_id;

/*Retrieve staff assignments for specific emergency events:*/

SELECT ee.event_name, st.name AS "Staff Name", sa.role

FROM Staff_Events sa

JOIN Staff st ON sa.staff_id = st.staff_id

JOIN Emergency_Events ee ON sa.event_id = ee.event_id;

/*Emergency Events Needing the Most Staff:*/

SELECT ee.event_name, COUNT(se.staff_id) AS "Total Staff Assigned"

FROM Staff_Events se

JOIN Emergency_Events ee ON se.event_id = ee.event_id

GROUP BY ee.event_name

ORDER BY COUNT(se.staff_id) DESC;

/*Retrieve Emergency Events with Dispatched Supplies(Inner Join)*/

SELECT ee.event_name, s.name AS Supply_Name,

ed.quantity_dispatched FROM Emergency_Events ee

INNER JOIN

Emergency_Distributions ed ON ee.event_id = ed.event_id

INNER JOIN

Supplies s ON ed.supply_id = s.supply_id;

/*List All Suppliers and Their Supply Orders*/

SELECT sp.name AS Supplier_Name, so.order_id, so.order_date,
so.quantity_ordered

FROM Suppliers sp

LEFT JOIN Supply_Orders so ON sp.supplier_id = so.supplier_id

ORDER BY sp.name, so.order_date;

/* Update salaries of staff based on their role:*/

Alter table Staff

add column salary decimal(10,3);

select * from Staff;

UPDATE Staff

SET salary = CASE

WHEN role = 'Warehouse Worker' THEN 10000.50

WHEN role = 'Supervisor' THEN 5000.00

WHEN role = 'Manager' THEN 3000.50

WHEN role = 'Inventory Specialist' THEN 7000.00

ELSE 2500

END;

SELECT* from Staff;

UPDATE Staff

SET salary = CASE

WHEN role = 'Manager' THEN salary * 1.10

ELSE salary * 1.05

END;

SELECT * from Staff;

/* Find events and their total dispatched supply cost:*/

```
SELECT ee.event_name, SUM(ed.quantity_dispatched * s.unit)
AS "Total Cost"
FROM Emergency_Distributions ed
JOIN Emergency_Events ee ON ed.event_id = ee.event_id
JOIN Supplies s ON ed.supply_id = s.supply_id
GROUP BY ee.event_name;
```

/*Check if a supply has been dispatched during any emergency event:*/

```
SELECT name,CASE
WHEN EXISTS (SELECT 1 FROM Emergency_Distributions ed
WHERE ed.supply_id = s.supply_id)
THEN 'Dispatched'
ELSE 'Not Dispatched'
END AS status FROM Supplies s;
```

/*Retrieve Supplies with the Highest Quantity in Each Category */

```
SELECT s1.name AS Supply_Name, s1.category, s1.reorder_level
FROM Supplies s1 WHERE
s1.reorder_level = (SELECT MAX(s2.reorder_level)
FROM Supplies s2 WHERE s2.category = s1.category)
ORDER BY s1.category;
```

/*Calculate Total Supplies Dispatched per Event */

```
SELECT ee.event_name,
SUM(ed.quantity_dispatched) AS Total_Dispatched
FROM Emergency_Events ee
INNER JOIN
```

```
Emergency_Distributions ed ON ee.event_id = ed.event_id
GROUP BY ee.event_name
ORDER BY Total_Dispatched DESC;
```

```
/*List Suppliers and the Number of Supplies They Provide*/
SELECT sp.name AS Supplier_Name, l.name AS Location_Name,
COUNT(sl.location_id) AS Locations_Served
FROM Suppliers sp
INNER JOIN
Supplier_Locations sl ON sp.supplier_id = sl.supplier_id
INNER JOIN
Locations l ON sl.location_id = l.location_id
GROUP BY sp.name, l.name
ORDER BY Locations_Served DESC;
```

```
/*Find the Total Quantity of Each Supply Dispatched Across All Events*/
SELECT s.name AS Supply_Name,
SUM(ed.quantity_dispatched) AS Total_Dispatched
FROM Supplies s Join
Emergency_Distributions ed ON s.supply_id = ed.supply_id
GROUP BY s.name ORDER BY Total_Dispatched ;
```

```
/*List Suppliers and Their Average Lead Time for Supplies They Provide*/
SELECT sp.name AS Supplier_Name,
AVG(sp.lead_time) AS Average_Lead_Time
FROM Suppliers sp
JOIN Supply_Orders so ON sp.supplier_id = so.supplier_id
GROUP BY sp.name
ORDER BY Average_Lead_Time ASC;
```

/*Regions With Highest Emergency Distributions*/

```
SELECT l.region,  
SUM(ed.quantity_dispatched) AS Total_Dispatched  
FROM Emergency_Distributions ed  
JOIN Locations l ON ed.location_id = l.location_id  
GROUP BY l.region ORDER BY Total_Dispatched DESC;
```

/*List Locations with Stock Levels Below Critical Thresholds*/

```
SELECT s.location_id,s.supply_id,s.quantity,sp.reorder_level  
FROM Stock s JOIN Supplies sp ON s.supply_id = sp.supply_id  
WHERE s.quantity < sp.reorder_level;
```

/*Predict Future Supply Needs Based on Past Usage Data*/

```
SELECT supply_id,  
AVG(quantity_dispatched) AS avg_quantity_dispatched  
FROM Emergency_Distributions GROUP BY supply_id;
```

/* Natural join */

```
select * from supplies Natural join Stock;
```

/* Add a Column */

```
ALTER TABLE Staff  
ADD COLUMN hire_date DATE;  
Select * from staff;
```

/* Delete a Column*/

```
ALTER TABLE Suppliers  
DROP COLUMN contact_info;
```

```
Select* from Suppliers;
```

```
/*      Update an Existing Record*/
```

```
UPDATE Supplies
```

```
SET reorder_level = 25
```

```
WHERE name = 'First Aid Kits';
```

```
Select * from supplies;
```